

Experiment Report

1. Objective

- Familiarize with basic Docker operations, including creating, starting, stopping, and removing containers.
 - Learn how to use Docker Desktop and Docker CLI to view containers, manage volumes, and control resources.
 - Learn to build custom images using Dockerfile and deploy a static webpage via Nginx.
 - Understand how to use Docker Compose to start multi-container applications.
- ## 2. Principle
- Docker is an open-source containerization platform used to package, distribute, and run applications.
- Image: A read-only template containing an application and its dependencies.
 - Container: A runnable instance of an image with read-write capabilities.
 - Volume: A mechanism for persisting data beyond the lifecycle of a container.
 - Dockerfile: A text file defining steps to build an image.
 - Docker Compose: Tool for defining and managing multi-container applications.

3. Environment

- Operating System: Windows 10/11
- Docker Version: Docker Desktop (latest)
- Terminal: PowerShell
- Browser: Any modern browser (for webpage verification)

4. Procedure

4.1 Start PostgreSQL Container

```
docker run --name db -e POSTGRES_PASSWORD=secret -d -v postgres_data:/var/lib/postgresql/data postgres
docker exec -ti db psql -U postgres
```

4.2 Create Table and Insert Data

```
CREATE TABLE tasks (
    id SERIAL PRIMARY KEY,
    description VARCHAR(100)
);
INSERT INTO tasks (description) VALUES ('Finish work'), ('Have fun');
SELECT * FROM tasks;
```

4.3 Stop and Remove Container

```
docker stop db
docker rm db
```

4.4 Use Volume for Persistent Data

```
docker run --name new-db -d -v postgres_data:/var/lib/postgresql/data postgres
docker exec -ti new-db psql -U postgres -c "SELECT * FROM tasks;"
```

4.5 Build Nginx Static Webpage Image 1. Create a public_html directory and add index.html (basic HTML page). 2. Create a Dockerfile:

```
FROM nginx:latest
COPY . /usr/share/nginx/html
EXPOSE 80
```

3. Build the image and run a container:

```
docker build -t nginx .
docker run --name web -d -p 8080:80 nginx
```

4.6 Verify Webpage Open a browser and navigate to <http://localhost:8080> to check the webpage.

5. Results (Insert screenshots here)

- Docker Desktop container list
- PostgreSQL table creation and query
- Volume persistence verification
- Nginx webpage access

6. Analysis and Discussion

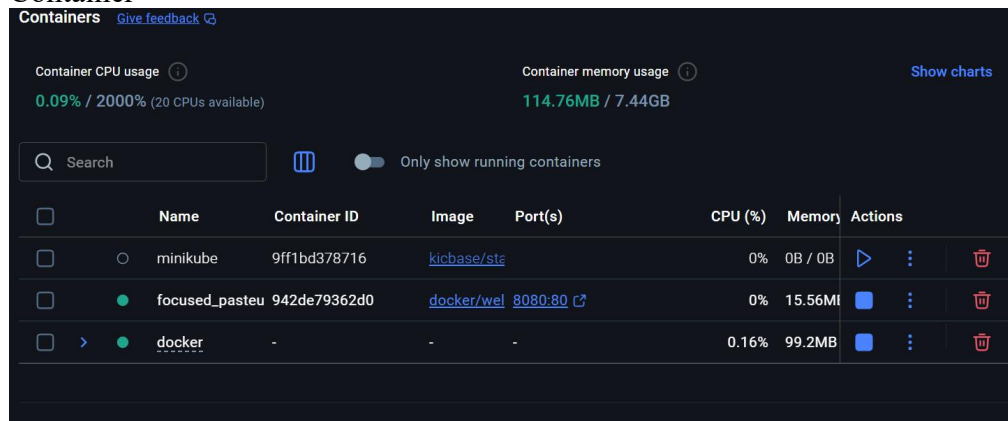
- Volumes allow data persistence even after the container is stopped or removed.
- Dockerfile makes it easy to package and deploy a static webpage quickly.
- Network issues may prevent pulling images; consider using a proxy or mirror registry.
- Docker Compose simplifies managing multi-container applications.

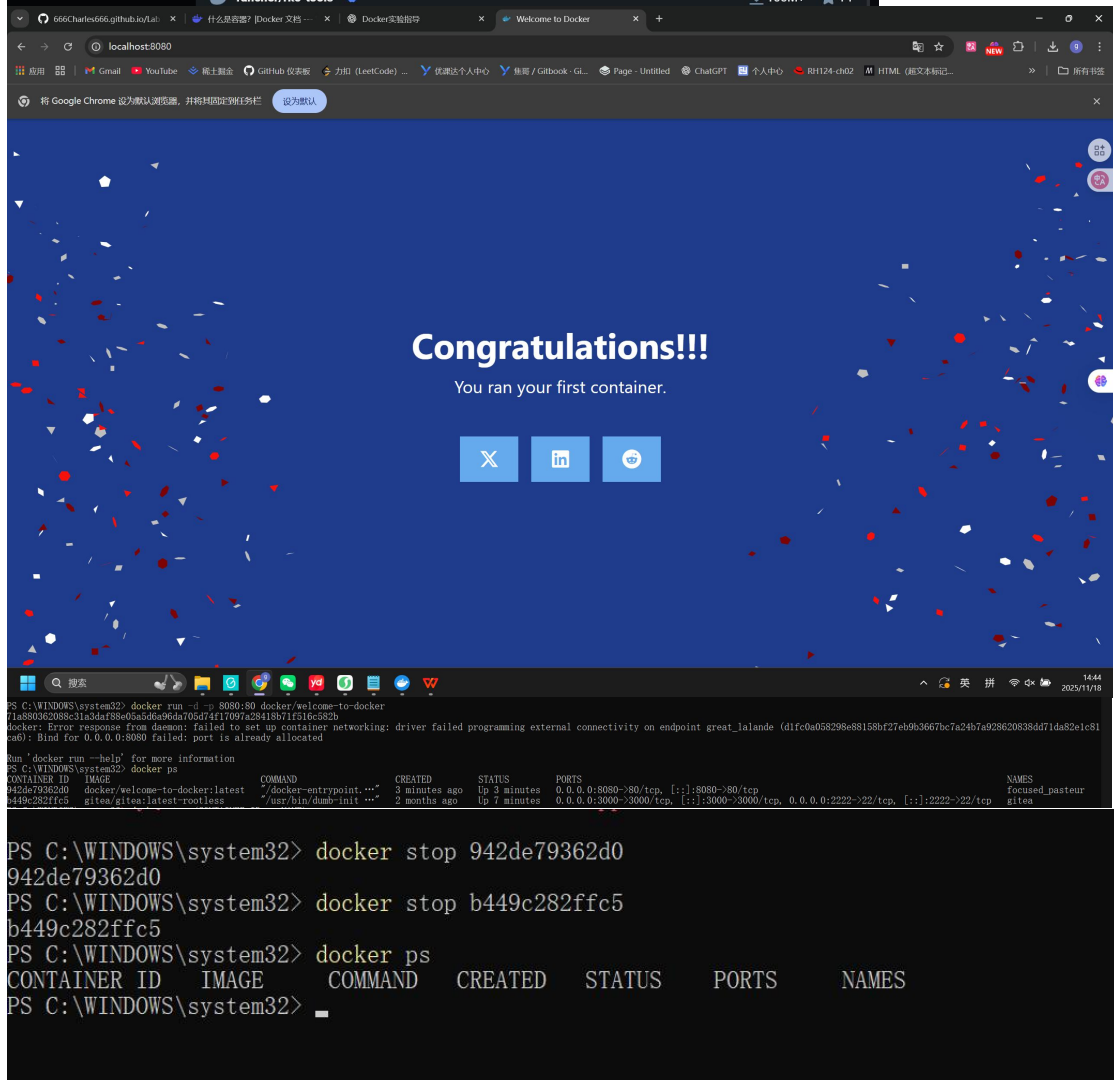
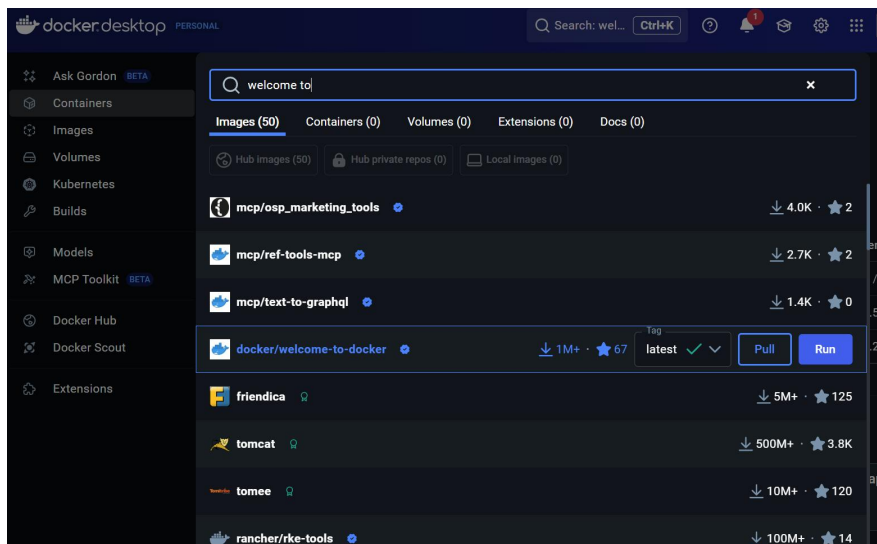
7. Conclusion

- Mastered basic Docker commands and container management.
- Understood the importance of volumes for data persistence.
- Successfully built and deployed a simple web service using Docker and Nginx.

8. Screenshot

Container

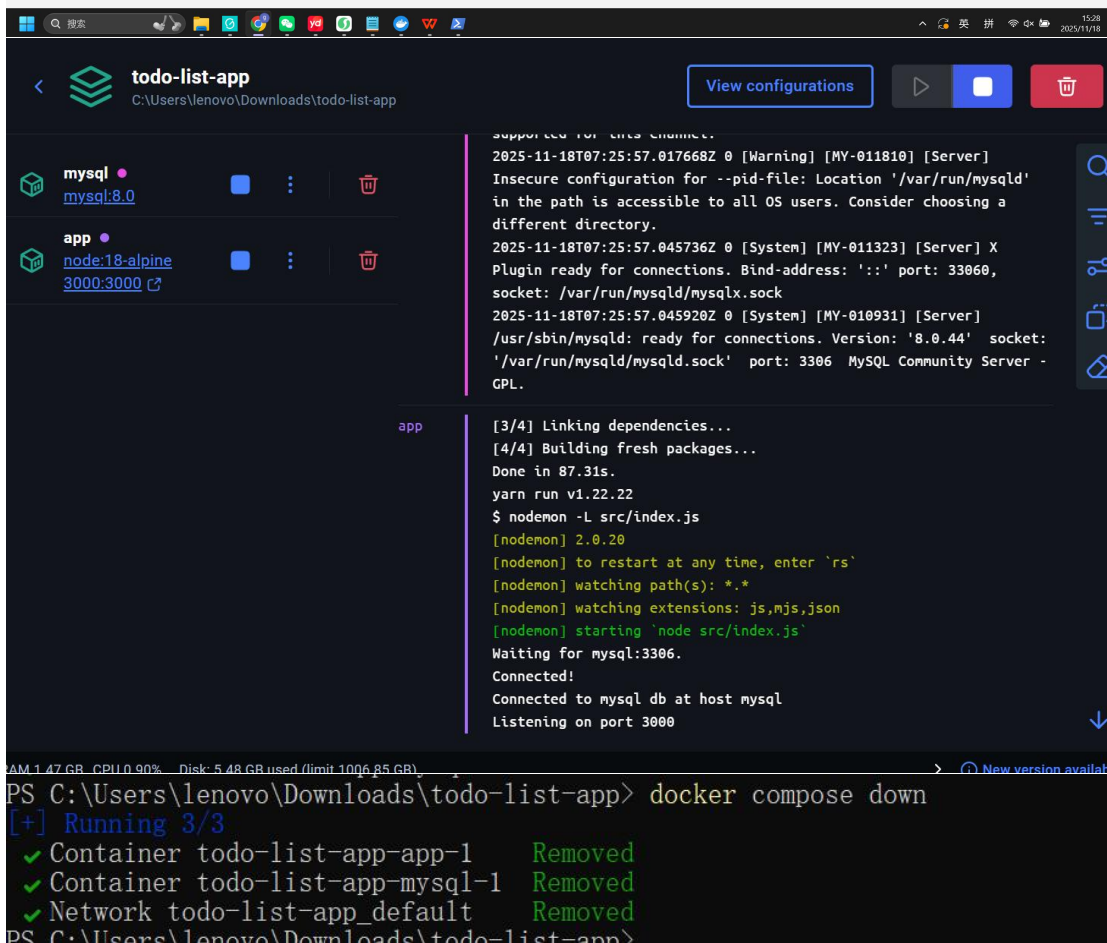
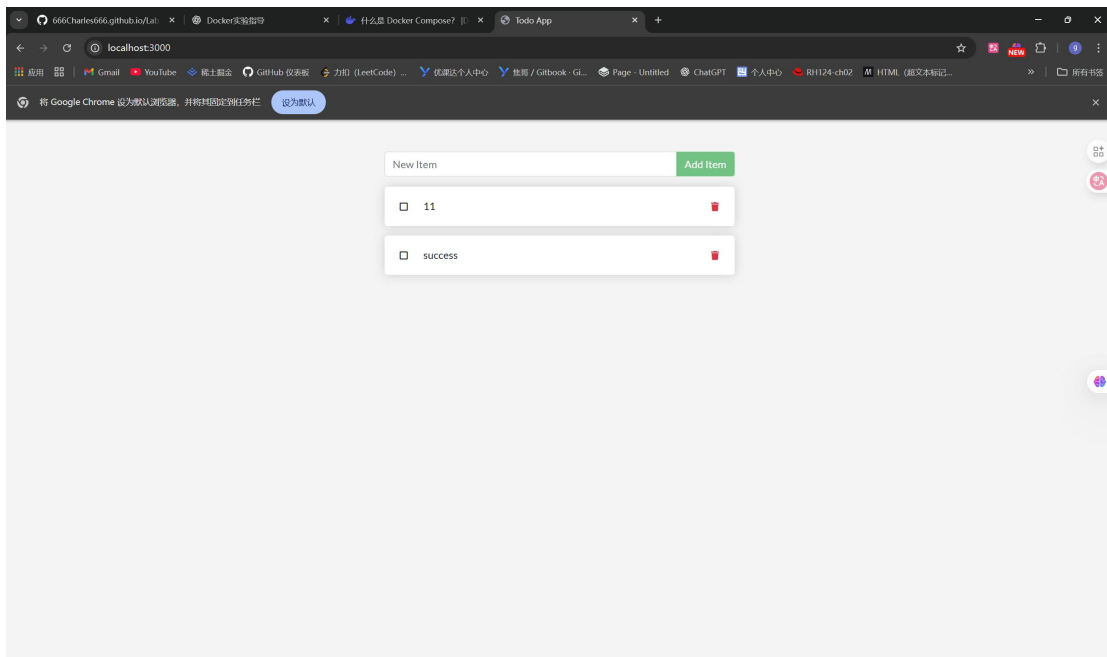




image

Docker compose

```
PS C:\Users\lenovo\Downloads> git clone https://github.com/docker-samples/todo-list-app
Cloning into 'todo-list-app'...
remote: Enumerating objects: 93, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 93 (delta 0), reused 0 (delta 0), pack-reused 91 (from 2)
Receiving objects: 100% (93/93), 1.68 MiB | 1.64 MiB/s, done.
Resolving deltas: 100% (15/15), done.
PS C:\Users\lenovo\Downloads> cd todo-list-app
PS C:\Users\lenovo\Downloads\todo-list-app> docker compose up -d --build
[+] Running 17/17
  ✓ app Pulled
    ✓ 25ff2da83641 Pull complete
    ✓ dd71dde834b5 Pull complete
    ✓ 1e5a4c89cee5 Pull complete
    ✓ f18232174bc9 Pull complete
  ✓ mysql Pulled
    ✓ a2ed1082d9e2 Pull complete
    ✓ 4f94eaa123bf Pull complete
    ✓ d68710a4a4e9 Pull complete
    ✓ c9ecfb07ed08 Pull complete
    ✓ 023a182c62a0 Pull complete
    ✓ fdca9f583d44 Pull complete
    ✓ 37bd516ff765 Pull complete
    ✓ 48ec49971d94 Pull complete
    ✓ 2a2d53254403 Pull complete
    ✓ 4f78e34adfad Pull complete
    ✓ abcf302dead6 Pull complete
[+] Running 4/4
  ✓ Network todo-list-app_default Created
  ✓ Volume "todo-list-app_todo-mysql-data" Created
  ✓ Container todo-list-app-app-1 Started
  ✓ Container todo-list-app-mysql-1 Started
```



Publishing and exposing ports

```
PS C:\Users\lenovo\Downloads\todo-list-app> docker run -d -p 8080:80 docker/welcome-to-docker
6ec4e0da8284f9d986401de5cd5605d4a3a50915f73a1c61fb8637fd9f687d86
```

	Name	Container ID	Image	Port(s)	CPU (%)	Memory	Actions
☐	minikube	9ff1bd378716	kicbase/sta		0%	0B / 0B	
☐	focused_pasteu	942de79362d0	docker/wel	8080:80	0%	0B / 0B	
☐	great_lalande	71a880362088	docker/wel	8080:80	0%	0B / 0B	
☐	determined_swæ	5bd3eff26094	docker/wel		0%	0B / 0B	
☐	cranky_herschel	2c275edd6375	docker/wel		0%	0B / 0B	
☐	magical_ramanj	6ec4e0da8284	docker/wel	8080:80	0%	16.59Mi	
☐	docker	-	-	-	0%	0B / 0B	

Congratulations!!!

You ran your first container.

```
PS C:\Users\lenovo> cd docker-compose-test
PS C:\Users\lenovo\docker-compose-test> notepad compose.yaml
PS C:\Users\lenovo\docker-compose-test> docker compose up -d
[+] Running 1/2
✔ Network docker-compose-test_default Created
- Container docker-compose-test-app-1 Starting
Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint docker-compose-test-app-1 (299c510513f9a5f86fa1230b0e7e0faba77124b5ef10a786d5a5d4a1497e7a588): Bind for 0.0.0.0:8080 failed: port is already allocated
PS C:\Users\lenovo\docker-compose-test> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
6ec4e0da8284   docker/welcome-to-docker   /docker-entrypoint.---  11 minutes ago Up 11 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   magical_ramanujan
PS C:\Users\lenovo\docker-compose-test> docker compose up -d
[+] Running 1/1
✔ Container docker-compose-test-app-1 Started
PS C:\Users\lenovo\docker-compose-test>
```

截屏工具

屏幕截图已复制到剪贴板
已自动保存到屏幕截图文件夹。

标记和共享

Overriding container defaults


```
PS C:\Users\lenovo\docker-compose-test> docker run -d -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres
Unable to find image 'postgres:latest' locally
latest: Pulling from library/postgres
1a50b0db5bde: Pull complete
cbf222e9e3b9: Pull complete
02119e61144b: Pull complete
3221278be714: Pull complete
24fdbfae116d: Pull complete
0e4bc2bd6656: Pull complete
5a19de04632f: Pull complete
cc81fb737f8b: Pull complete
c6aa08a3c711: Pull complete
57871086ddd6: Pull complete
d75755489c86: Pull complete
fed2b3560f55: Pull complete
889840adelab: Pull complete
Digest: sha256:8ce6e01f36d23849cf5a92275a94baec9ce28f32a47cd729375afb63ca6f985d
Status: Downloaded newer image for postgres:latest
a1f4dd2ddfc6ae804a30d60cda7ac92c49769e08c8a5965d697b235a5287c31f
PS C:\Users\lenovo\docker-compose-test> docker run -d -e POSTGRES_PASSWORD=secret -p 5433:5432 postgres
453e2351bee536829ba8219e2fb2fdc3e418d238c12ab7864f466bb98b97bfe8
```

		eager_sanderso	a1f4dd2ddfc6		postgres	5432:5432		0.03%	27.43M			
		quizzical_moore	453e2351bee5		postgres	5433:5432		0.07%	23.75M			

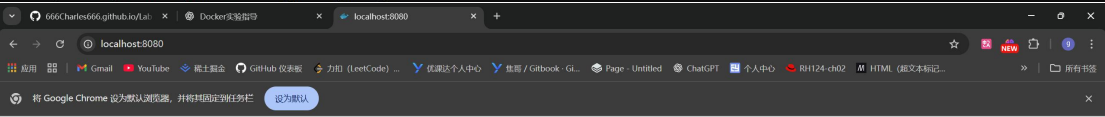
```
✓ Container docker-compose-test-app-1 started
PS C:\Users\lenovo\docker-compose-test> docker run -d -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres
Unable to find image 'postgres:latest' locally
latest: Pulling from library/postgres
1a50b0db5bde: Pull complete
cbf222e9e3b9: Pull complete
02119e61144b: Pull complete
3221278be714: Pull complete
24fdbfae116d: Pull complete
0e4bc2bd6656: Pull complete
5a19de04632f: Pull complete
cc81fb737f8b: Pull complete
c6aa08a3c711: Pull complete
57871086ddd6: Pull complete
d75755489c86: Pull complete
fed2b3560f55: Pull complete
889840adelab: Pull complete
Digest: sha256:8ce6e01f36d23849cf5a92275a94baec9ce28f32a47cd729375afb63ca6f985d
Status: Downloaded newer image for postgres:latest
a1f4dd2ddfc6ae804a30d60cda7ac92c49769e08c8a5965d697b235a5287c31f
PS C:\Users\lenovo\docker-compose-test> docker run -d -e POSTGRES_PASSWORD=secret -p 5433:5432 postgres
453e2351bee536829ba8219e2fb2fdc3e418d238c12ab7864f466bb98b97bfe8
PS C:\Users\lenovo\docker-compose-test> docker network create mynetwork
7977e3a98810aa034742ec4503f481a53c783736ff10872865fb0d21e35c2673
PS C:\Users\lenovo\docker-compose-test> docker network ls
NETWORK ID          NAME                DRIVER            SCOPE
40701ae73ad0        bridge              bridge            local
9d0a7b11bca1        docker-compose-test_default    bridge            local
22ba2ae26037        docker_default      bridge            local
e3ae92e6d043        host                host              local
4ad57f04b78a        minikube            bridge            local
7977e3a98810        mynetwork           bridge            local
9a72bf3b7047        none                null              local
PS C:\Users\lenovo\docker-compose-test> docker run -d -e POSTGRES_PASSWORD=secret -p 5434:5432 --network mynetwork postgres
e391fabafaf5add063a4294f76119b7544896858a3058b52fcbdb03fe3c884b91
PS C:\Users\lenovo\docker-compose-test> docker network inspect
docker: 'docker network inspect' requires at least 1 argument

Usage: docker network inspect [OPTIONS] NETWORK [NETWORK...]

See 'docker network inspect --help' for more information
PS C:\Users\lenovo\docker-compose-test>
```



```
PS C:\Users\lenovo\docker-compose-postgres> docker run -d -p 8080:80 --name my_site httpd:2.4
Unable to find image 'httpd:2.4' locally
2.4: Pulling from library/httpd
87a14f083967: Pull complete
4742a9e996d1: Pull complete
5b4d5959fc75: Pull complete
4f4fb700ef54: Pull complete
9cd0271fa751: Pull complete
Digest: sha256:3b41f31c39c9a73221116d2a0e549561fb6483124b21b4daa2155be5ffe72d0f
Status: Downloaded newer image for httpd:2.4
e5b9a9df5d241ec9cf86e1675cf705720c8c013ec8460434b7856f2f8b7fa921
PS C:\Users\lenovo\docker-compose-postgres>
```



It works!

```
PS C:\Users\lenovo\docker-compose-postgres> docker run -d -p 8080:80 --name my_site httpd:2.4
Unable to find image 'httpd:2.4' locally
2.4: Pulling from library/httpd
87a14f083967: Pull complete
4742a9e996d1: Pull complete
5b4d5959fc75: Pull complete
4f4fb700ef54: Pull complete
9cd0271fa751: Pull complete
Digest: sha256:3b41f31c39c9a73221116d2a0e549561fb6483124b21b4daa2155be5ffe72d0f
Status: Downloaded newer image for httpd:2.4
e5b9a9df5d241ec9cf86e1675cf705720c8c013ec8460434b7856f2f8b7fa921
PS C:\Users\lenovo\docker-compose-postgres> mkdir public_html

目录: C:\Users\lenovo\docker-compose-postgres

Mode                LastWriteTime         Length Name
----                -
d-----          2025/11/18      16:14         public_html

PS C:\Users\lenovo\docker-compose-postgres> cd public_html
PS C:\Users\lenovo\docker-compose-postgres\public_html> nano index.html
nano: 无法将 "nano" 识别为 cmdlet、函数、脚本文件或可运行程序的名称。请检查名称的拼写，如果包括路径，请将其
所在位置 行:1 字符:1
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (nano:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\lenovo\docker-compose-postgres\public_html> New-Item -Name index.html -ItemType File

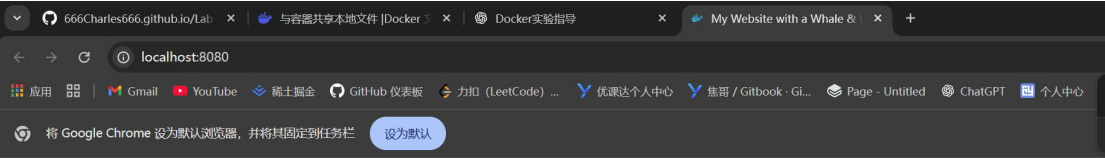
目录: C:\Users\lenovo\docker-compose-postgres\public_html

Mode                LastWriteTime         Length Name
----                -
-a-----          2025/11/18      16:15              0 index.html

PS C:\Users\lenovo\docker-compose-postgres\public_html> notepad index.html
PS C:\Users\lenovo\docker-compose-postgres\public_html>
```

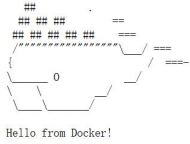
```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title> My Website with a Whale & Docker!</title>
</head>
<body>
<b>Whalacommell</b>
<p>Look! There's a friendly whale greeting you!</p>
<pre id="docker-art">
##      .
### ### ==
### ### ## ==
/===== \
/         \
{          /====
 \         \
  \         \
   \         \

Hello from Docker!
</pre>
</body>
</html>
```

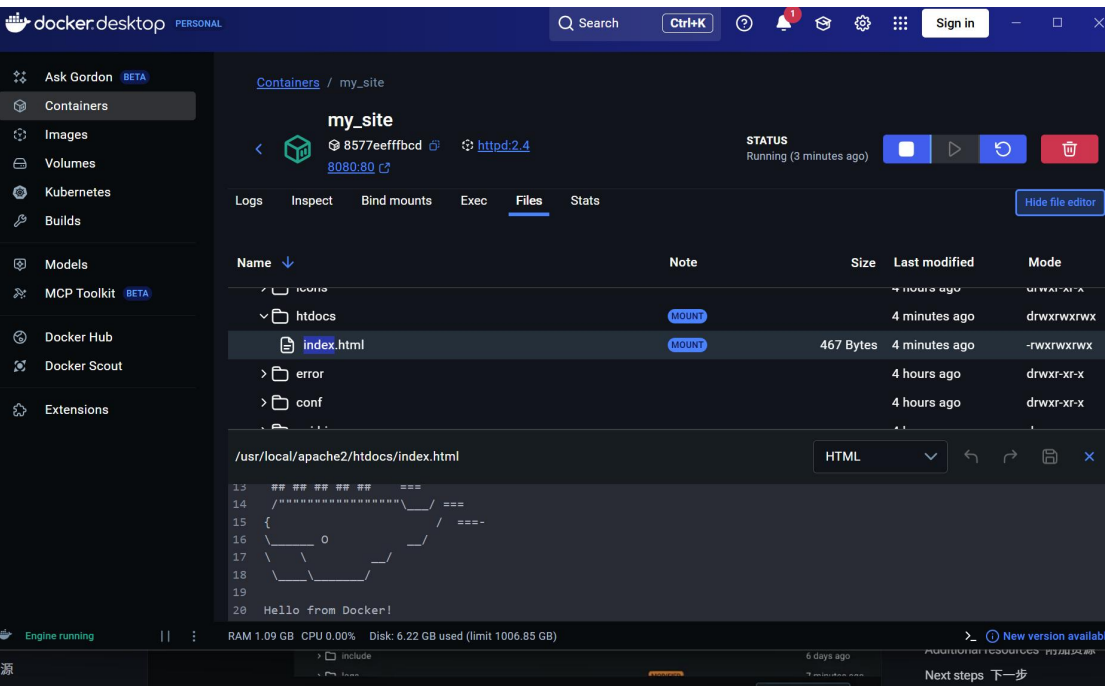


Whalecome!!

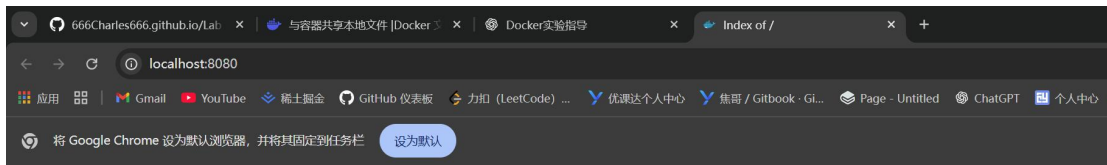
Look! There's a friendly whale greeting you!



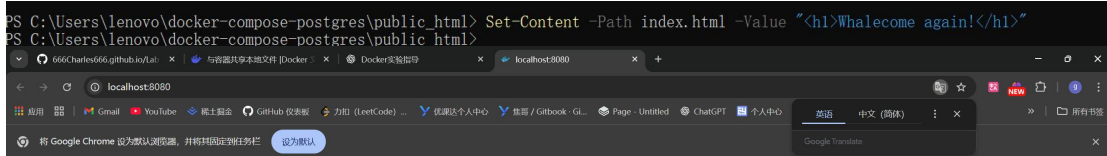
Hello from Docker!



When delete

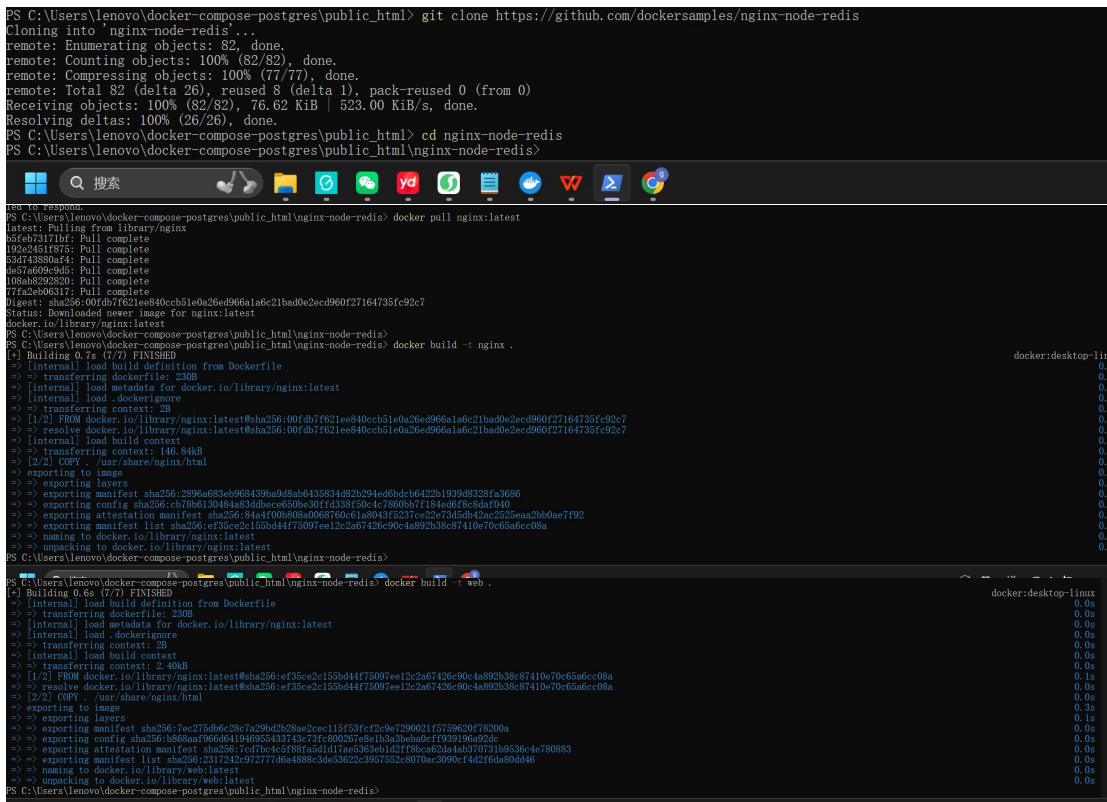


Index of /



Whalecome again!

Multi-container applications



Container	Image	IP Address	Port	CPU	Memory	Actions
☐	my_site	8577eaffbcbd	httpd:2.4 8080:80 ↗	0.01%	8.76MB	
☐	redis	ef9e33158dc9	redis	0.43%	4.84MB	
☐	web1	459096270ee9	web	0%	19.82M	
☐	web2	6f9d20511280	web	0%	15.82M	
☐	nginx	314cf39834ff	nginx 80:80 ↗	0%	16.06M	
☐	docker	-	-	0%	0B / 0B	